TELECOM SudParis

IP PARIS

Ontologies and Description Logic

Amel Bouzeghoub

What is Description Logic

A family of formal (logic-based) knowledge representation languages to represent the knowledge of an application domain in a

structured and well- understood way.

TELECOM SudParis

Why the name DL?

- Application domain (world) are described by concept ***description*** i.e expressions are built from atomic concepts (classes), atomic roles (relationships) and individuals.

- Equipped with formal ***logic*** based semantics

TELECOM SudParis

Why DL rather than Predicate Logic?

- DL has variable-free syntax.
- Easier to read than corresponding Predicate logic formulae.
- Combine means of expressiveness with decidability of the important reasoning problems.
- Most DL are decidable fragments of FOL designed for decidability and efficient reasoning.
- FOL: expressive but undecidable.

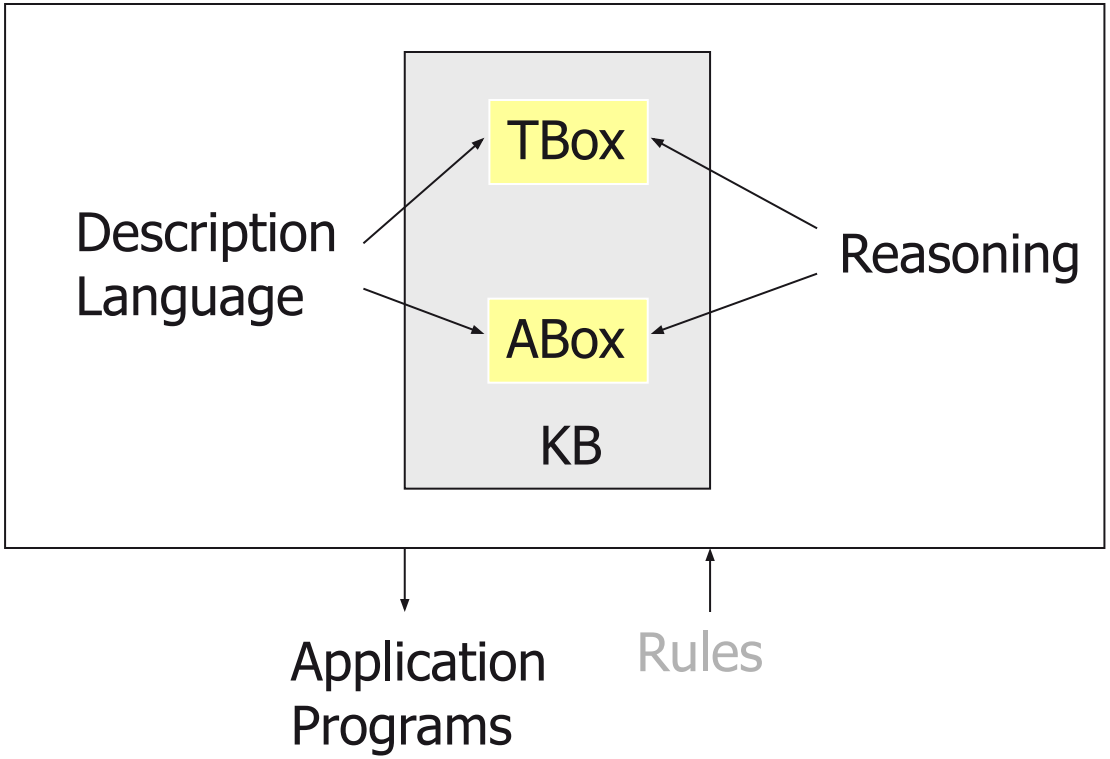
About Description Logics

- Description Logics (DLs) - knowledge representation formalisms that
  - represent the knowledge of an application domain (the world)
  - by defining the relevant **concepts** of the domain (its terminology)
  - and then using these concepts to specify properties of **individuals** occurring in the domain (the world description).
- Inferences: classification of
  - concepts - subconcept/superconcept relationships (subsumption)
  - individuals - whether a given individual is always an instance of a concept

A DL system

TBox – the terminology, i.e., the vocabulary of an application domain
= concepts (set of individuals) + roles

ABox – assertions about named individuals in terms of this vocabulary



TBox:
Mother ≡ Woman ⊓ ∃ hasChild.⊤
ABox:
Woman(Alice), hasChild(Alice, Bob)
⇒ Mother(Alice)

Inference services

- **Typical reasoning tasks:**
  - Whether a TBox description is **satisfiable** (i.e., non-contradictory)
  - Whether one description **subsumes** another one in a TBox - organize the concepts of a terminology into a hierarchy according to their generality
  - Find out whether the set of assertions in a ABox is **consistent** (has a model)
  - Whether the assertions in the ABox entail that a particular individual is an **instance of** a given concept description
  - A concept description can also be conceived as a **query** - retrieve the individuals that satisfy the query.

TELECOM SudParis

Syntax of DL Concept Description

A, B, C, D → atomic concepts

R → atomic role

Basic constructors (AL):

- ⊤ : universal concept (all individuals) (Top concept)
- ⊥ : bottom concept (no individuals)
- ¬A : atomic negation (only for atomic concepts)
- C ⊓ D : intersection
- ∀R.C : value restriction (all R-successors are in C)
- ∃R.⊤ : limited existential quantification

Extensions:

- ALU: adds union (C ⊔ D)
- ALE: adds full existential restriction (∃R.C)
- ALC: adds full negation (¬C)
- ALN: adds number restrictions (≤n R, ≥n R)

AL ⊆ ALE ⊆ ALC ⊆ SHOIN ⊆ SROIQ (OWL 2 DL)

Examples

DL Expression	Meaning
$\mathtt{Person} \sqcap \mathtt{Female}$	Females who are persons
$\exists \mathtt{hasChild}.\top$	Those who have at least one child (of any type)
$\exists \mathtt{hasChild}.\mathtt{Person}$	Has at least one child who is a person
$\mathtt{Person} \sqcap \exists \mathtt{hasChild}.\top$	Persons who have at least one child
$\mathtt{Person} \sqcap \forall \mathtt{hasChild}.\mathtt{Female}$	Persons whose all children are female
$\mathtt{Person} \sqcap \forall \mathtt{hasChild}.\bot$	Persons with no children
$\mathtt{Person} \sqcap \mathtt{Female} \sqcap \geq 3\ \mathtt{hasChild}$	Female persons with at least three children

Semantics

Semantics of DL

- An **interpretation** $\mathbb{I} = (\Delta^I, \cdot^I)$ consists of:
  - a **domain** Δ^I (non-empty set)
  - an **interpretation function** $\cdot^I$ that maps:
    - each atomic concept A → a subset $A^I \subseteq \Delta^I$
    - each atomic role R → a relation $R^I \subseteq \Delta^I \times \Delta^I$
- The semantics of complex expressions are defined recursively:
  - $(C \sqcap D)^I = C^I \cap D^I$
  - $(\neg C)^I = \Delta^I \setminus C^I$
  - $(\forall R.C)^I = \{ x \in \Delta^I \mid \forall y, (x,y) \in R^I \rightarrow y \in C^I \}$
  - $(\exists R.C)^I = \{ x \in \Delta^I \mid \exists y, (x,y) \in R^I \wedge y \in C^I \}$

Example

Let's interpret the concept:

$\text{Parent} \equiv \text{Person} \sqcap \exists\, \text{hasChild.Person}$

Semantically:

$\text{Parent}^I = \{x \in \Delta^I \mid x \in \text{Person}^I \text{ and } \exists\, y.(x,y) \in \text{hasChild}^I \wedge y \in \text{Person}^I\}$

→ "The set of all individuals who are persons and have at least one child who is also a person."

In FOL:

$\forall x.\text{Parent}(x) \leftrightarrow (\text{Person}(x) \wedge \exists\, y.(\text{hasChild}(x,y) \wedge \text{Person}(y)))$

Terminologies (TBox)

- Defines relationships between concepts (and roles).
- Axioms:
  - Inclusion: $C \sqsubseteq D$ (“C is a subclass of D”)
  - Equivalence: $C \equiv D$ (“C and D have the same instances”)
- If T is a set of axioms, then I satisfies T iff I satisfies each element of T
  – I is a **model** of T
  - $\mathbb{I} \models T$ iff $\forall (C \sqsubseteq D \text{ or } C \equiv D) \in T$, $\mathbb{I} \models$ that axiom. So a model of T is an interpretation in which **all** the inclusions and equivalences in T hold.
  - **Example:**
  - If T = { Parent $\sqsubseteq$ Person, Mother $\equiv$ Woman $\sqcap$ $\exists$ hasChild.Person },
  - then $\mathbb{I}$ is a *model of T* if both those relationships are true in $\mathbb{I}$.

Example of TBox

$\mathbf{Woman} \equiv \mathbf{Person} \sqcap \mathbf{Female}$

$\mathbf{Man} \equiv \mathbf{Person} \sqcap \neg\mathbf{Woman}$

$\mathbf{Mother} \equiv \mathbf{Woman} \sqcap \exists\, \mathbf{hasChild}.\mathbf{Person}$

$\mathbf{Father} \equiv \mathbf{Man} \sqcap \exists\, \mathbf{hasChild}.\mathbf{Person}$

$\mathbf{Parent} \equiv \mathbf{Father} \sqcup \mathbf{Mother}$

$\mathbf{Grandmother} \equiv \mathbf{Mother} \sqcap \exists\, \mathbf{hasChild}.\mathbf{Parent}$

$\mathbf{MotherWithManyChildren} \equiv \mathbf{Mother} \sqcap \geq 3\ \mathbf{hasChild}$

$\mathbf{MotherWithoutDaughter} \equiv \mathbf{Mother} \sqcap \forall\, \mathbf{hasChild}.\neg\mathbf{Woman}$

$\mathbf{Wife} \equiv \mathbf{Woman} \sqcap \exists\, \mathbf{hasHusband}.\mathbf{Man}$

World descriptions (ABox)

- ABox introduce individuals, by giving them names, and asserts properties of these individuals
- Describe a concrete situation by stating properties of individuals
  - Individuals: a, b, c
  - C(a) - concept assertions - a belongs to (the interpretation of) C
  - R(b,c) - role assertions - c is a filler of the role R for b.

MotherWithoutDaughter(Mary)
Father(Peter)
hasChild(Mary, Peter)
hasChild(Peter, Harry)
hasChild(Mary, Paul)

ABox Semantics

- An interpretation I satisfies C(a) if $a^I \in C^I$
- An interpretation I satisfies R(a,b) if $(a^I, b^I) \in R^I$
- An interpretation I satisfies an ABox if it satisfies each assertion in the ABox – I is a model of A
- An interpretation I satisfies an *ABox A with respect to a TBox T* if it is a model of T and a model of A

Reasoning tasks for TBox

- Satisfiability: A concept C is satisfiable with respect to T if there exists a model I of T such that C^I is nonempty (I is a *model* of C)
- Subsumption: A concept C is subsumed by a concept D with respect to T if $C^I \subseteq D^I$ for every model I of T

  $C \sqsubseteq D$ or $T \models C \sqsubseteq D$.

- Equivalence: Two concepts C and D are equivalent with respect to T if $C^I = D^I$ for every model I of T

  $C \equiv_T D$ or $T \models C \equiv D$

- Disjointness: Two concepts C and D are disjoint with respect to T if $C^I \cap D^I = \varnothing$ for every model I of T

Properties

Reduction to Subsumption

- C is unsatisfiable ⇔ C is subsumed by ⊥
- C and D are equivalent ⇔ C is subsumed by D and D is subsumed by C
- C and D are disjoint ⇔ C ⊓ D is subsumed by ⊥

Reduction to Unsatisfiability

- C is subsumed by D ⇔ C ⊓ ¬D is unsatisfiable
- C and D are equivalent ⇔ both (C ⊓ ¬D) and (¬C ⊓ D) are unsatisfiable
- C and D are disjoint ⇔ C ⊓ D is unsatisfiable

Reasoning tasks for ABox

1. Consistency of an ABox
   a. An ABox A is consistent with respect to a TBox T, if $\exists \mathcal{I}$ that is a model of both A and T

Father $\sqsubseteq$ Man, Mother $\sqsubseteq$ Woman, Man $\sqcap$ Woman $\sqsubseteq \bot$ # a person cannot be both a man and a woman

Mother(Mary), Mary$^{\mathcal{I}} \in$ Mother$^{\mathcal{I}} \subseteq$ Woman$^{\mathcal{I}}$

Father(Mary), Mary$^{\mathcal{I}} \in$ Father$^{\mathcal{I}} \subseteq$ Man$^{\mathcal{I}}$

Man $\sqcap$ Woman $\sqsubseteq \bot$, Man$^{\mathcal{I}} \cap$ Woman$^{\mathcal{I}} = \varnothing \Rightarrow$ A is inconsistent w.r.t. T

   b. A is consistent if it is consistent with respect to the empty TBox.

{ Mother(mary), Father(mary) }

Reasoning tasks for ABox

2. Instance check
   a. An assertion C(a) is entailed by A if ∀ I that satisfies A also satisfies C(a) (A|= C(a))

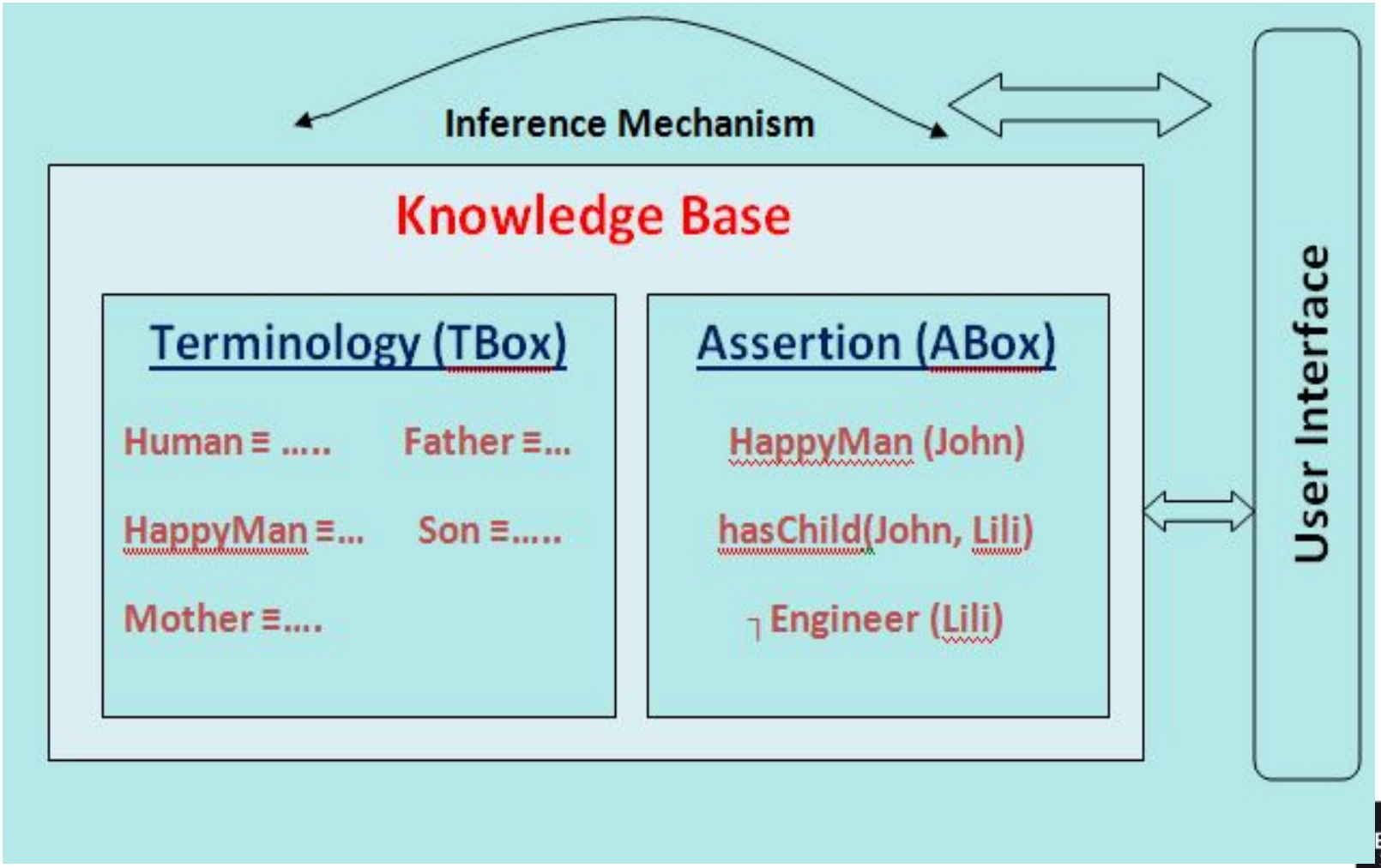
      A|= C(a) iff A ∪ {¬C(a)} is inconsistent

3. Retrieval problem
   a. Given an ABox A and a concept C, find all individuals *a* such that A|= C(a)

4. Realization problem
   a. Given an individual *a* and a set of concepts, find the most specific concepts C from the set such that A|= C(a)

      The most specific concepts are those that are minimal with respect to the subsumption ordering

DL Architecture



Important Extensions of ALC

- We may want to define a HappyMan concept with the instance of having children between two and four

$$\text{HappyMan} \equiv \text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor})) \sqcap (\geq 2\,\text{hasChild} \sqcap \leq 4\,\text{hasChild})$$

- Qualified number restrictions:

  The definition of the HappyMan concept can be extended by including the instances of the having at least two children who are professors.

$$\text{HappyMan} \equiv \text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor})) \sqcap (\geq 2\,\text{hasChild}.\text{Professor} \sqcap \leq 4\,\text{hasChild})$$

Important Extensions of ALC cont.

- Inverse roles:
  - the role hasChild has a inverse role has.Parent.
- Transitive roles:
  - a role is transitive, e.g to capture hasAncestor
- Subroles:
  - one role is a subrole of another role, e.g hasParent is a subrole of hasAncestor
- Concrete domains:
  - Integrate DL with Real numbers, Integers, Strings, Concrete predicates

TELECOM SudParis

DLs and First Order Predicate Logic (FOL)

DL Syntax	Meaning	FOL Equivalent
$\top$, $\bot$	universal / empty concept	$\forall x.\top$, $\forall x.\bot$
$\neg C$	negation	$\neg C(x)$
$C \sqcap D$	conjunction	$C(x) \wedge D(x)$
$C \sqcup D$	disjunction	$C(x) \vee D(x)$
$C \sqsubseteq D$		$\forall x.\ C(x) \rightarrow D(x)$
$\exists R.C$	existential restriction	$\exists y.\ R(x,y) \wedge C(y)$
$\forall R.C$	universal restriction	$\forall y.\ R(x,y) \rightarrow C(y)$
$R1 \circ R2 \sqsubseteq R3$	Property chain	$\forall x,y,z.\ R1(x,y) \wedge R2(y,z) \rightarrow R3(x,z)$

TELECOM SudParis

From DL to OWL

OWL as a DL-based ontology language

OWL = Web Ontology Language.

Built on DL semantics (e.g., OWL 2 DL = SROIQ(D)).

Allows automated reasoning on web ontologies.

OWL reasoners (HermiT, Pellet, FaCT++) Integrated in Protégé
In Python: Owlready2, owlrl



Ontologies

What is an ontology ?

- No exact definition
- A tool to help organize knowledge
- Or a way to convey a theory on how to represent a class of things
- The term ontology is originated from philosophy.

  the study of the nature of existence.

Definitions

- Webster's Third New International Dictionary defines Ontology
  - A science or study of being specifically, a branch of metaphysics relating to the nature and relations of being.
  - A theory concerning the kinds of entities and specifically the kinds of abstract entities that are to be admitted to a language system.
- An ontology in **Computer Sciences**
  - « explicit formal specifications of the terms in the domain and relations among them » (Gruber 1993)
  - Semantic Relations
  - Composition and inheritance relationships

Definition (Hudelot 2006)

- An ontology is a formal and explicit specification of a shared conceptualization of a field of knowledge.
- Conceptualization: a certain view of the world in relation to a domain, often conceived as a set of concepts, their definition, their interrelationships.
- Explicit: explicit definition of the types of concepts used and constraints on their use.
- Formal: understandable by the machine.
- Shared: consensus, knowledge accepted by a group.

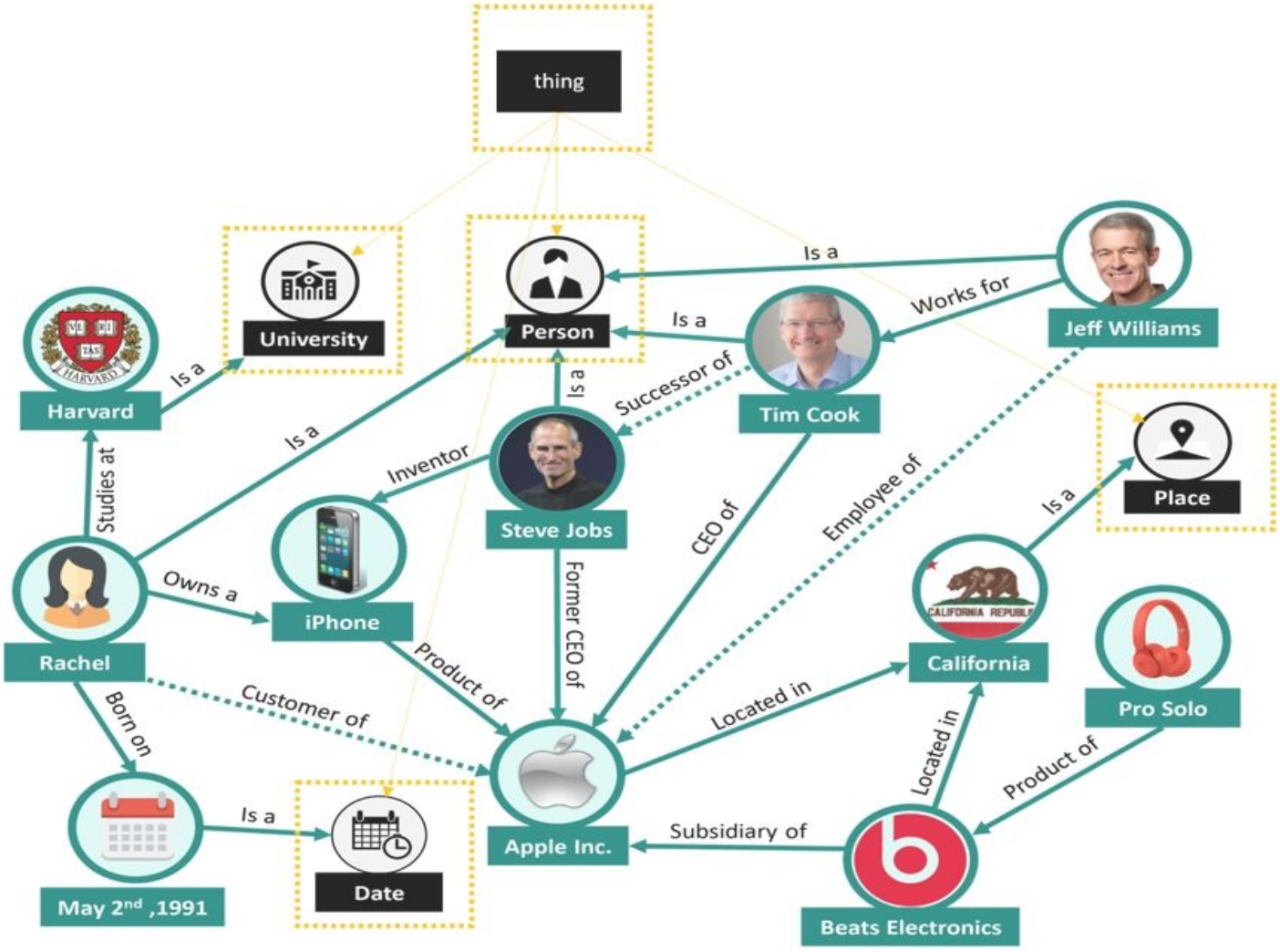
The Role of an Ontology

- Basic Role
  - To provide a language which allows a group of people to share information reliably in a chosen area of work
  - Reuse
    - Create and maintain reusable knowledge bases
    - Build knowledge bases from reusable modules
  - Knowledge sharing and communication
    - Ensure interoperability between systems
    - Enable knowledge exchange between systems

Why do I need to develop an ontology?

- To provide more precise definition to resources and make them more suitable to machine processing
- To separate domain knowledge from operational knowledge
  - Re-use domain and operational knowledge separately
- A common reference to define concepts and relationships between concepts and other axioms in a domain
- To share a consistent understanding of a domain
- Reasoning / inference
  - Building and discovering new information and/or knowledge from existing ontologies and resources
- To support explainable and trustworthy AI

Example



https://timbr.ai/introducing-timbr-sql-kg/

TELECOM SudParis

Knowledge Graphs vs Ontologies

- **A Knowledge Graph stores data as a graph**
  - A knowledge graph = data organized as triples (subject–predicate–object).
  - The Knowledge Graph is the instance level (ABOX)

- **But a Knowledge Graph needs meaning → that comes from the Ontology**
  - Ontology = the schema / vocabulary / logic of the knowledge graph.
  - The Ontology is the concept level (TBOX)

- **The Ontology enables reasoning on the Knowledge Graph**

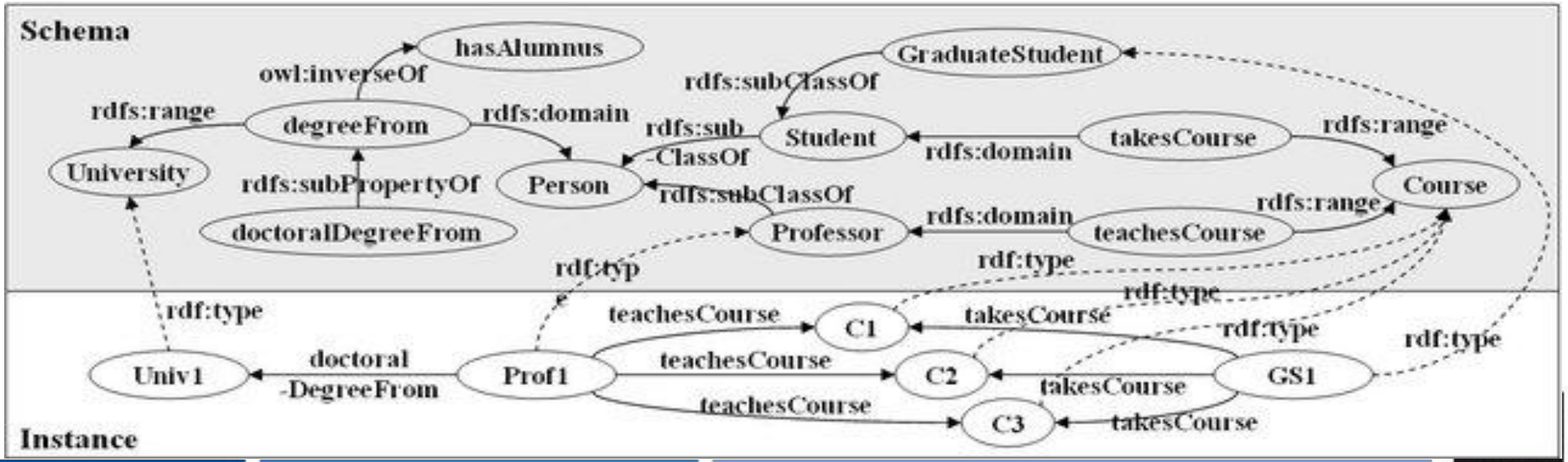
TELECOM SudParis

Ontology Structure

- **An ontology structure is a quintuplet O := {** C, $\mathcal{R}$, $\mathcal{H}^C$, $\mathcal{R}el$, $\mathcal{A}^O$ **}**
  - C and $\mathcal{R}$: disjointed sets of concepts and relations
  - $\mathcal{H}^C$ **hierarchy (taxonomy) of concepts** : $\mathcal{H}^C \subseteq C \times C$, $\mathcal{H}^C(C_1, C_2)$ means that C_1 is a sub-concept of C_2 (oriented relation)
  - $\mathcal{R}el$: **relation** rel: $\mathcal{R} \rightarrow C \times C$ (defines non-taxonomic semantic relations) with 2 associated functions
    - $dom : \mathcal{R} \rightarrow C$ with $dom(\mathcal{R}) := \prod 1(rel(\mathcal{R}))$
    - $range : \mathcal{R} \rightarrow C$ with $range(\mathcal{R}) := \prod 2(rel(\mathcal{R}))$ co-domaine
    - $rel(\mathcal{R}) = (C_1, C_2)$ or $\mathcal{R}(C_1, C_2)$
  - $\mathcal{A}^O$: set of axioms, expressed in an adapted logical language (description logic, 1st order logic)

Knowledge Base

- **Structure of a knowledge base : quadruplet** $\mathcal{KB} := \{O, I, inst, instr\}$
  - $O := \{C, \mathcal{R}, \mathcal{H}^C, rel, \mathcal{A}^O\}$ is an ontology
  - I is a set of **individual**
  - $inst : C \rightarrow 2^I$ Concept instantiation
  - $Instr : \mathcal{R} \rightarrow 2^{I \times I}$ Relationship instantiation
- Lexicon of a knowledge base $\mathcal{L}^{\mathcal{KB}} := (\mathcal{L}^I, \mathcal{J})$

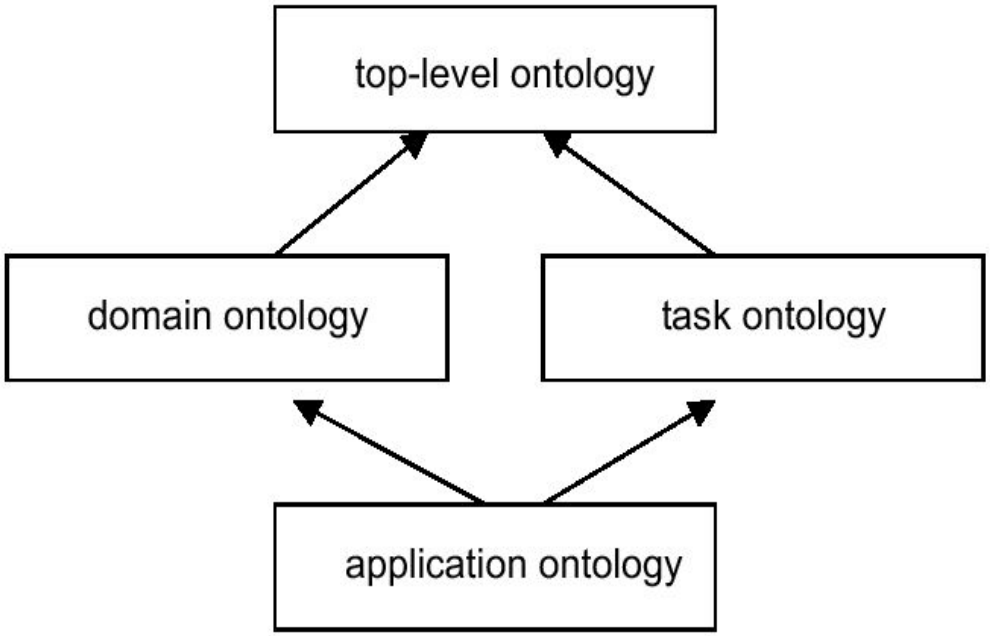


Different types of ontologies

[Guarino, 98]

Describe **very general concepts** like space, time, event, which are independent of a particular problem or domain. It seems reasonable to have unified top-level ontologies for large communities of users.

Describe the vocabulary related to a **generic domain** by specializing the concepts introduced in the top-level ontology.



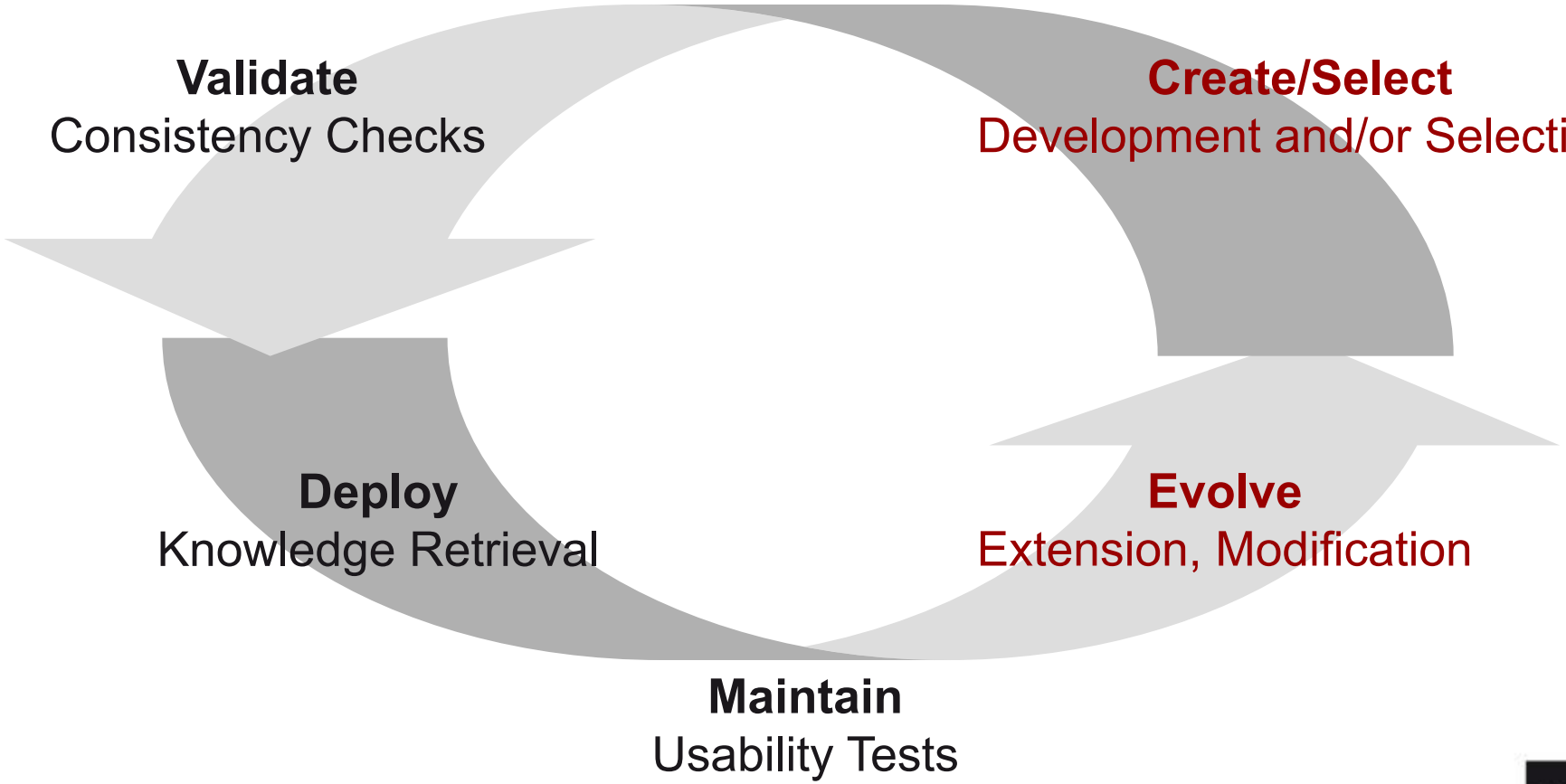
Describe the vocabulary related to a **generic task or activity** by specializing the top-level ontologies.

These are the most specific ontologies. Concepts in application ontologies often correspond to **roles played by domain entities while performing a certain activity**.

Heavyweight vs. Lightweight

- They differ in their expressiveness, reasoning capacity, complexity, decidability.
  - Lightweight
    - E-R diagrams, UML
  - Heavyweight
    - Description logic (DL), 1st order logic
- There are standards for each case (RDF, RDF Schema, OWL)

Life cycle of an ontology



Reasoning: rules of inference

- Modus ponens

$$\frac{A \;,\; A \rightarrow B}{B}$$

- Modus tollens

$$\frac{\neg B \;,\; A \rightarrow B}{\neg A}$$

-
- …

Reasoning: the basic cycle

- **Detection**
  - Determines the rules and relevant facts by means of "pattern matching" unifications
- **Choice**
  - Decides which of the applicable rules should actually be triggered
- **Execution**
  - Executes the action part of the rule.
  - Updates the Knowledge base.

The selection phase

- **Selects a rule from the set of applicable rules**
- **Methods:**
  - The first applicable rule (e.g. Prolog)
  - The most specific rule
  - The most useful rule (according to a utility value to be calculated)
- **The metarules**
  - Rules controlling the selection of rules to be applied

The triggering phase

- Three inference algorithms :

$$\frac{\mathbf{A}\ ,\ A \rightarrow B}{\mathbf{B}}$$

  - Forward chaining (data driven)
    - to deduce a particular fact: the rules whose premises are known are triggered - up to - the fact to be deducted is also known - or that no rule can be triggered any more
  - Backward chaining (goal oriented)

$$\frac{\mathbf{B}\ ,\ A \rightarrow B}{\mathbf{A}}$$

    - start from the fact to prove,
    - search for all the rules that conclude on that fact,
    - draw up a list of facts that need only be proved in order for them to be triggered and then
    - apply recursively the same mechanism to the facts contained in those lists
    - These facts in turn become facts to be proven.
    - Recursion stops when the goal to prove belongs to the fact base
  - Mixed chaining

Forward Chaining Example

R1: B et D et E → F
R2: D et G → A
R3: C et F → A
R4: C → D
R5: D → E
R6: A → H
R7: B → X
R8: X et C → A

B True
C True
H ?

Strategy 1: the first rule

B, C ____ B, C, D ____ B, C, D, E ____ B, C, D, E, F ____
B, C, D, E, F, A ____ B, C, D, E, F, A, H

Strategy 2: the last rule

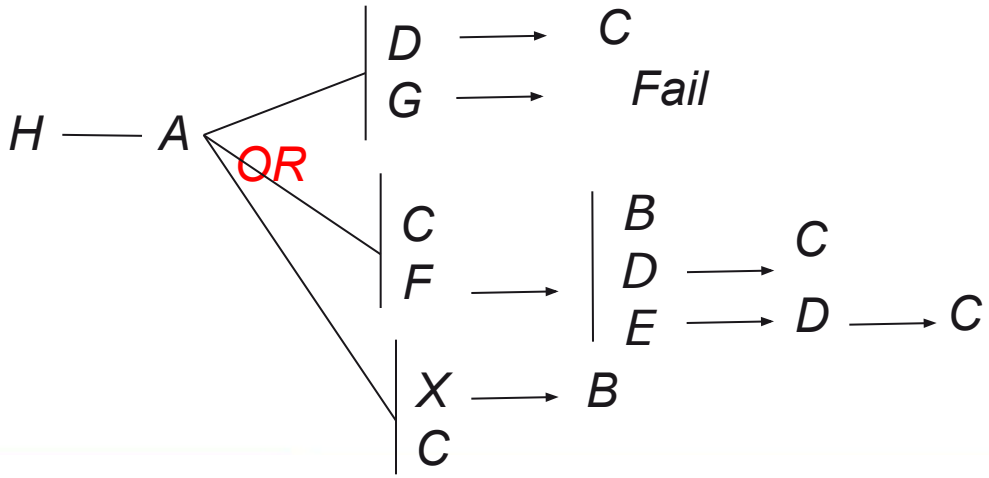
B, C ____ B, C, X ____ B, C, X, A ____ B, C, X, A, H

TELECOM SudParis

Backward Chaining Example

R1: B et D et E → F
R2: D et G → A
R3: C et F → A
R4: C → D
R5: D → E
R6: A → H
R7: B → X
R8: X et C → A

B True
C True
H ?



Example

The law states that it is a crime for an American to sell arms to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.

⇒ Let's prove that West is a criminal

Example

“... it is a crime for an American to sell arms to hostile nations.”:

- American(x) ∧ Arm(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal(x)

“Nono ... has some missiles”:i.e ∃ x Missile(x) ∧ Owns(Nono, x)

- Own(Nono, M1) ∧ Missile (M 1)

“.. all of its missiles were sold to it by Colonel West” :

- ∀x Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono)

Missiles are armes :

- ∀x Missile(x) ⇒ Arme(x)

An enemy of America count as « hostile » :

- ∀x Enemy(x, America) ⇒ Hostile(x)

“…West, who is American”:

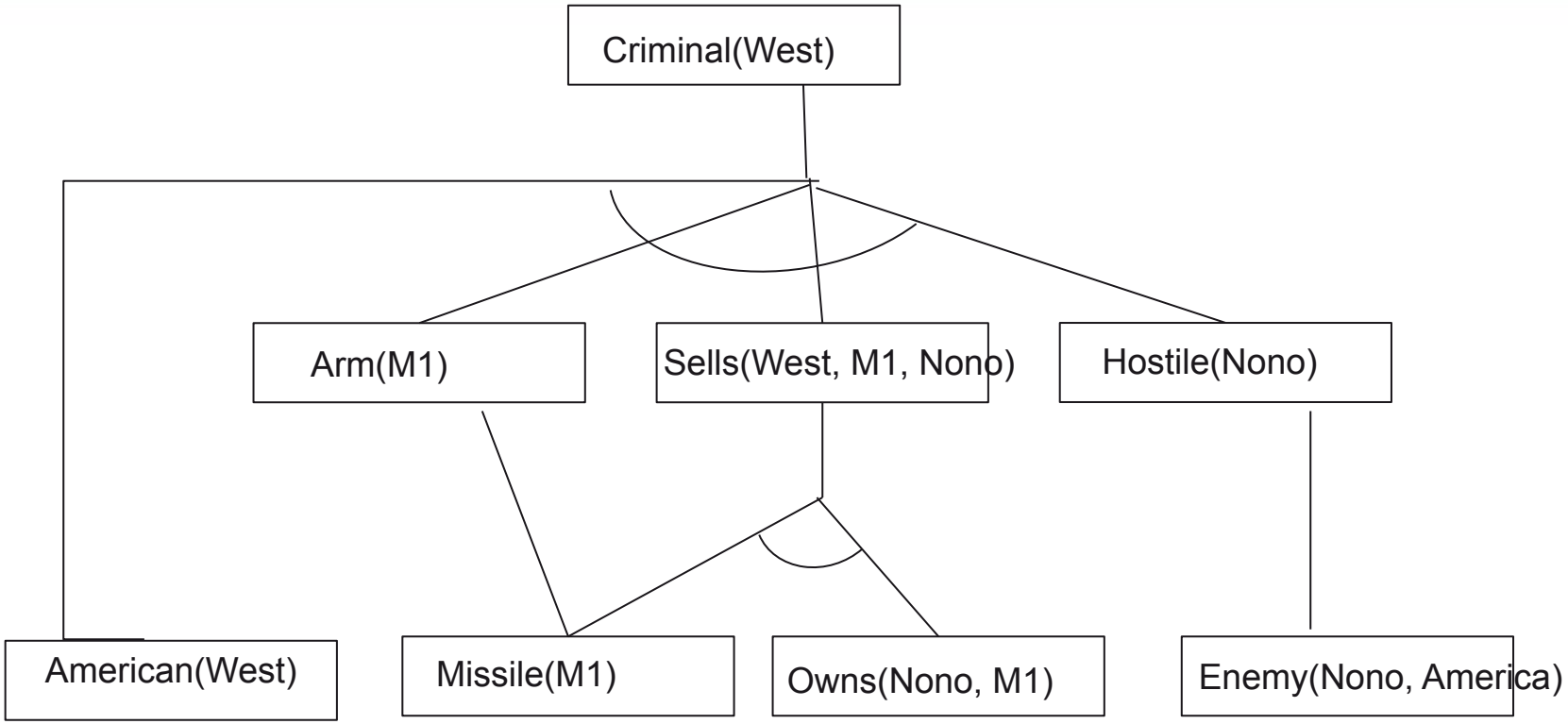
- American(West)

“The country Nono, an enemy of America” :

- Enemy(Nono, America)

TELECOM SudParis

Forward Chaining Proof



American(x) ∧ Arm (y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)

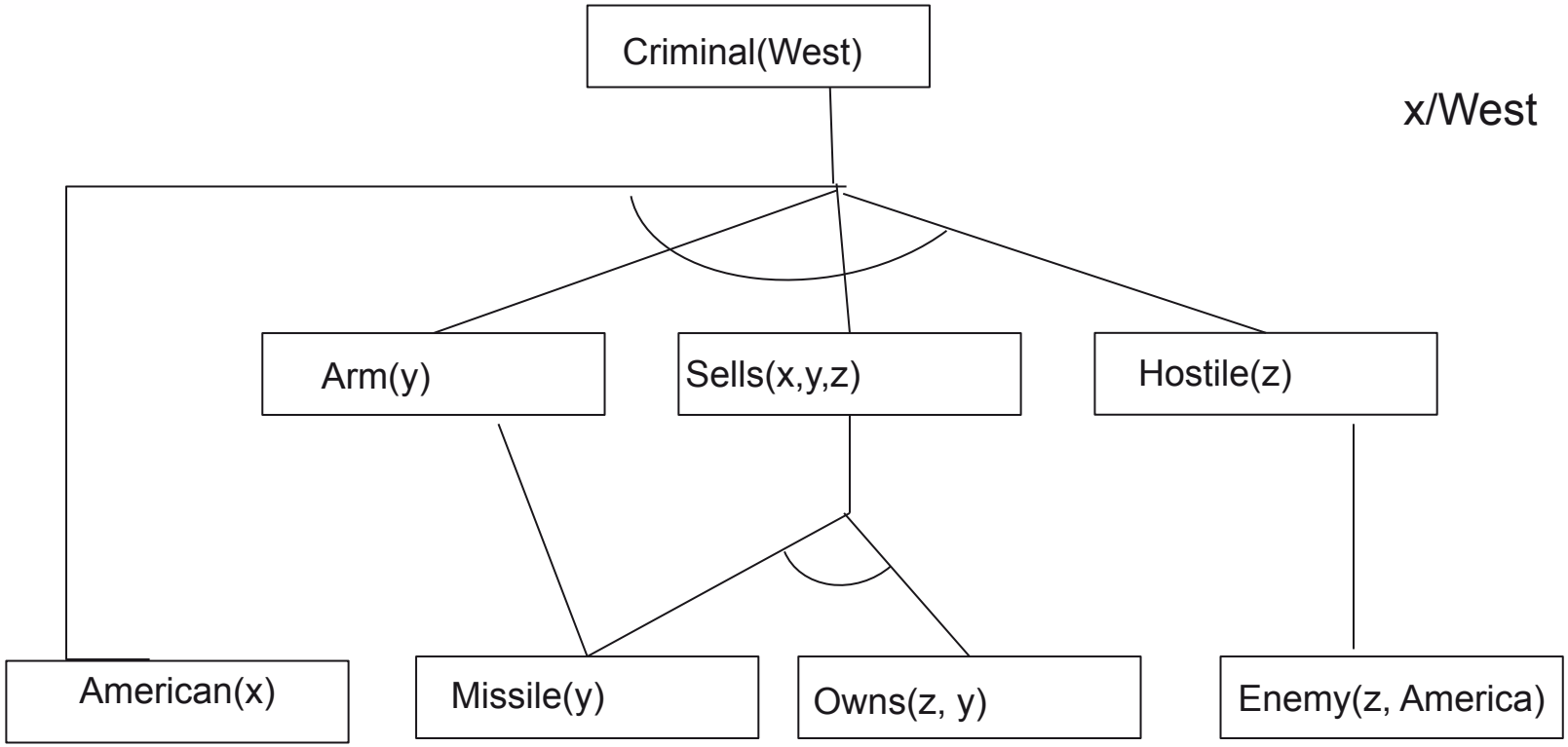
Owns(Nono,M_1) ∧ Missile(M_1)

Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)

Missile(x) ⇒ Arme) Enemy(x,America) ⇒ Hostile(x)

American(West) Enemy(Nono,America)

Backward Chaining Proof



American(x) ∧ Arm (y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)

Owns(Nono,M_1) ∧ Missile(M_1)

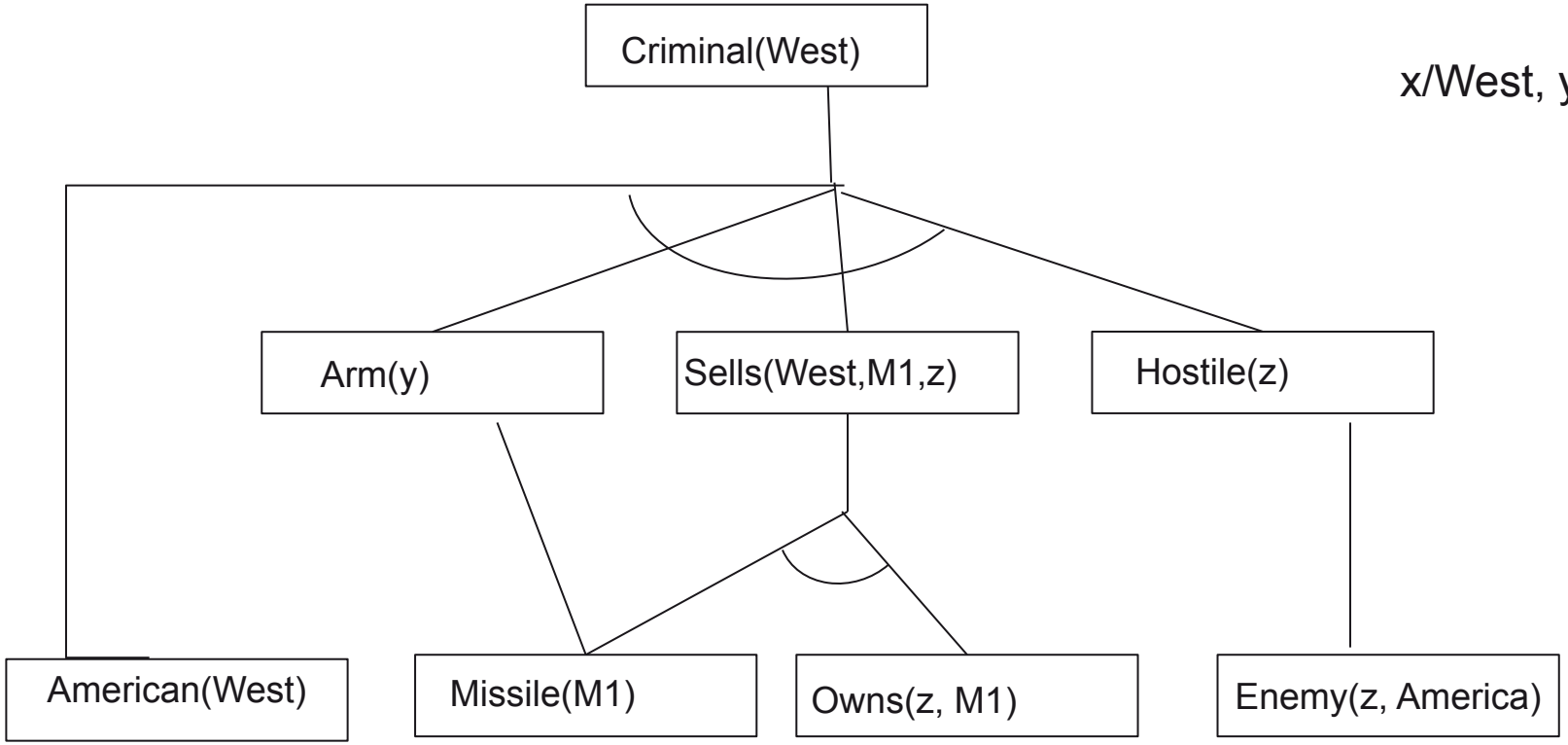
Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)

Missile(x) ⇒ Arme) Enemy(x,America) ⇒ Hostile(x)

American(West) Enemy(Nono,America)

TELECOM SudParis

Backward Chaining Proof



American(x) ∧ Arm (y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)

Owns(Nono,M_1) ∧ Missile(M_1)

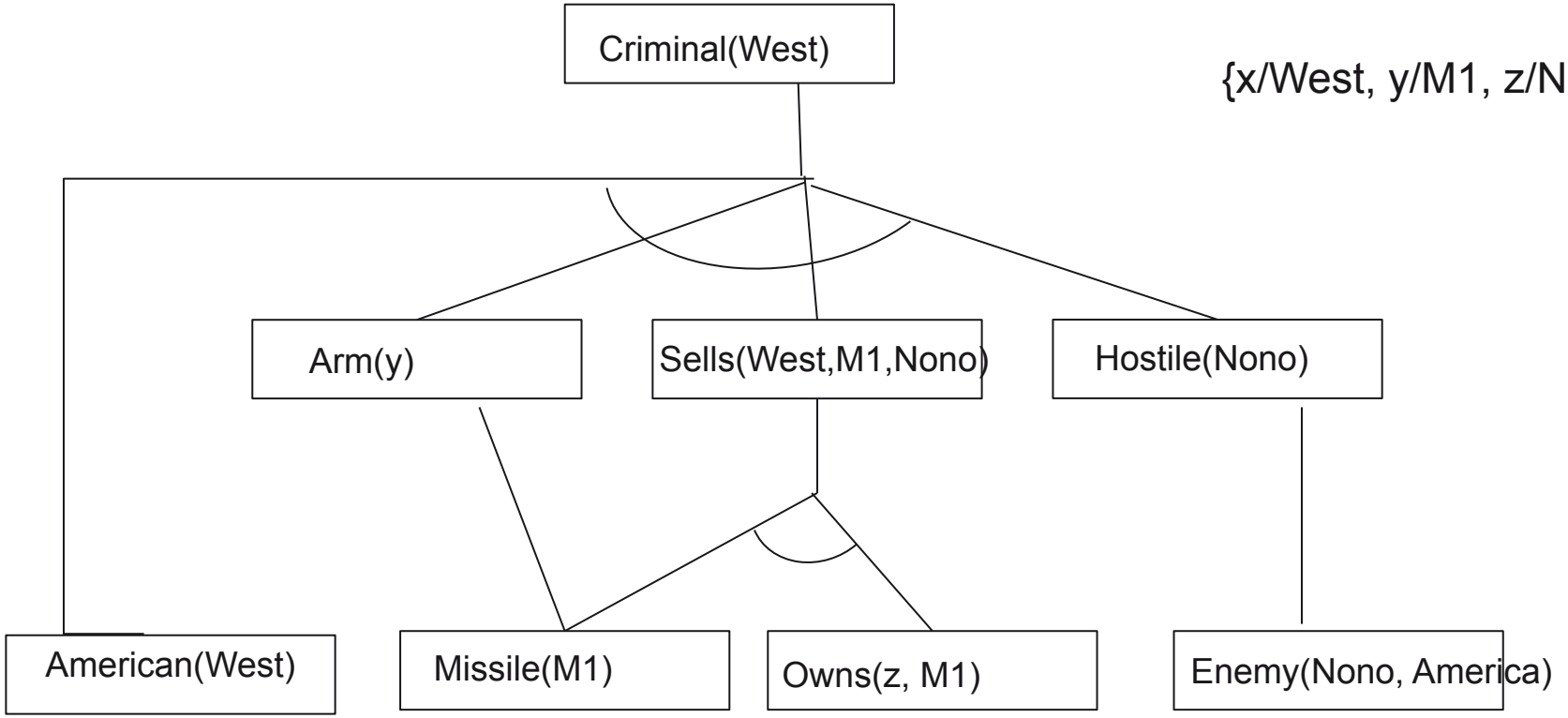
Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)

Missile(x) ⇒ Arme) Enemy(x,America) ⇒ Hostile(x)

American(West) Enemy(Nono,America)

TELECOM SudParis

Backward Chaining Proof



American(x) ∧ Arm (y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)

Owns(Nono,M_1) ∧ Missile(M_1)

Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)

Missile(x) ⇒ Arme) Enemy(x,America) ⇒ Hostile(x)

American(West) Enemy(Nono,America)

TELECOM SudParis

IP PARIS

OWL

Introduction

- is based on research carried out in the field of description logic
- used to describe ontologies, i.e. it enables terminologies to be defined to describe concrete domains
- is an important step forward in the representation and organisation of knowledge available on the Web
- is designed as an extension of the Resource Description Framework (RDF) and RDF Schema (RDFS).

Introduction

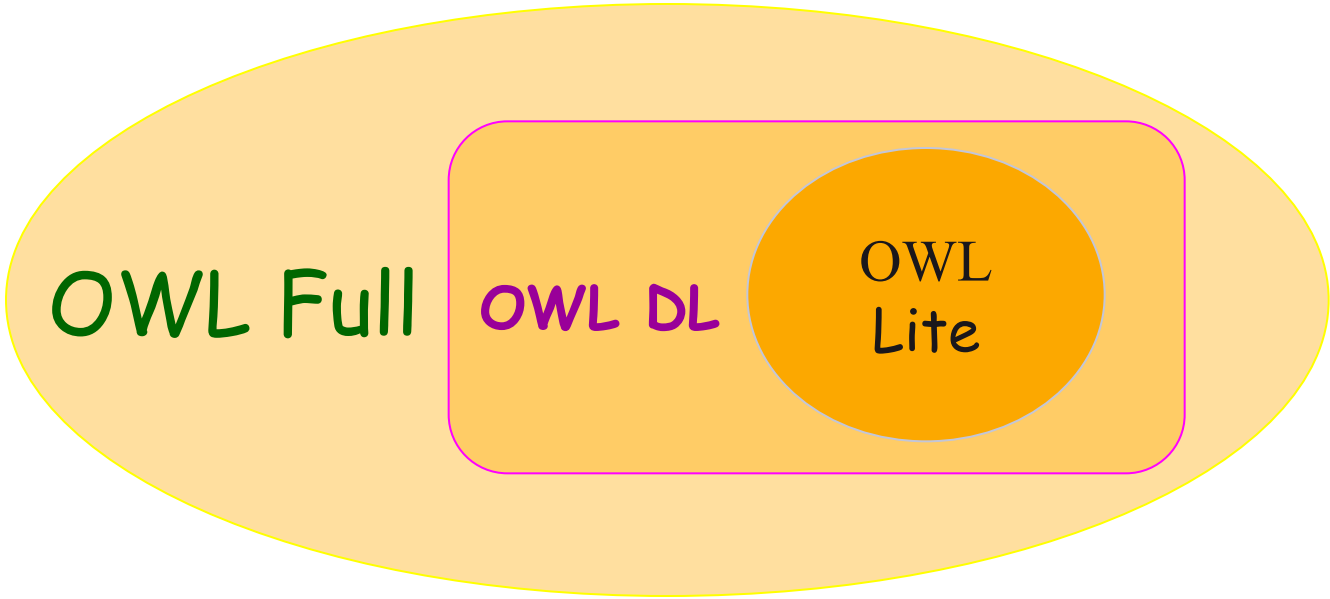
- **RDF and RDFs alone are too limited :**
  - Cannot specify the nature of the relationships between resources (reflexivity, etc.)
  - No capacity for reasoning
  - Very limited logic
- **The need for OWL :**
  - Derives from RDF + RDFS
  - Logical connectors between classes (union, intersection, etc.)
  - Cardinality on properties
  - Characterization of properties (transitivity, inverse, etc.)



Main advantages

- Brings better integration, evolution, sharing and easier inference of ontologies
- Adds the concepts of equivalent classes, equivalent properties, equality of two resources, their differences, the opposite, symmetry and cardinality
- Thanks to its formal semantics based on a widely studied logical foundation, allows to define more complex associations of resources as well as the properties of their respective classes
- Is suitable for the Semantic Web, as it offers a strictly defined syntax, and depending on the level can allow automated reasoning on knowledge inferences and conclusions

OWL sub-languages



Definition of OWL classes

- **A class can be declared in several ways:**
  - By naming the class or,
  - By enumeration of its individuals
  - By restricting the properties of its individuals
  - By intersection (AND), union (OR) or complement (NOT) of another class

  ==> Anonymous Classes: The members of an anonymous class are the set of Individuals that satisfy its logical definition
- **There is an inheritance mechanism** (<owl:subClassOf>)
- **The superclass** owl:Thing **is the mother of all the other classes**
- owl:Nothing **is subclass of all classes**
- **In OWL Full, a class can be an instance of another class (a "**metaclass**").**
- **The set of instances of a class is called "the extension".**

Restriction Types

∃	Existential, someValuesFrom	“Some”, “At least one”
∀	Universal, allValuesFrom	“Only”
∋	hasValue	“equals x”
=	Cardinality	“Exactly n”
≤	Max Cardinality	“At most n”
≥	Min Cardinality	“At least n”

Property Characteristics

- **Domain and range can be set**
- **OWL offers a mechanism for property inheritance:**

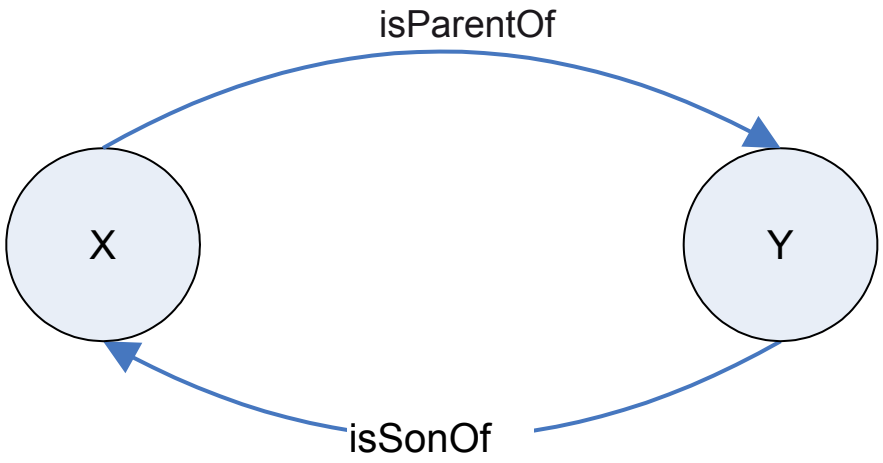
owl:ObjectProperty rdf:ID="hasBrother">
<rdfs:subPropertyOf rdf:resource="#isSibling" />
<rdfs:range rdf:resource="#Person" />
<rdfs:domain rdf:resource="#Person" />
</owl:ObjectProperty>

- **Properties can be characterized:**
  - Inverse
  - Transitive
  - Symetric
  - Functional
  - Inverse Functional

isParentOf

Inverse Property

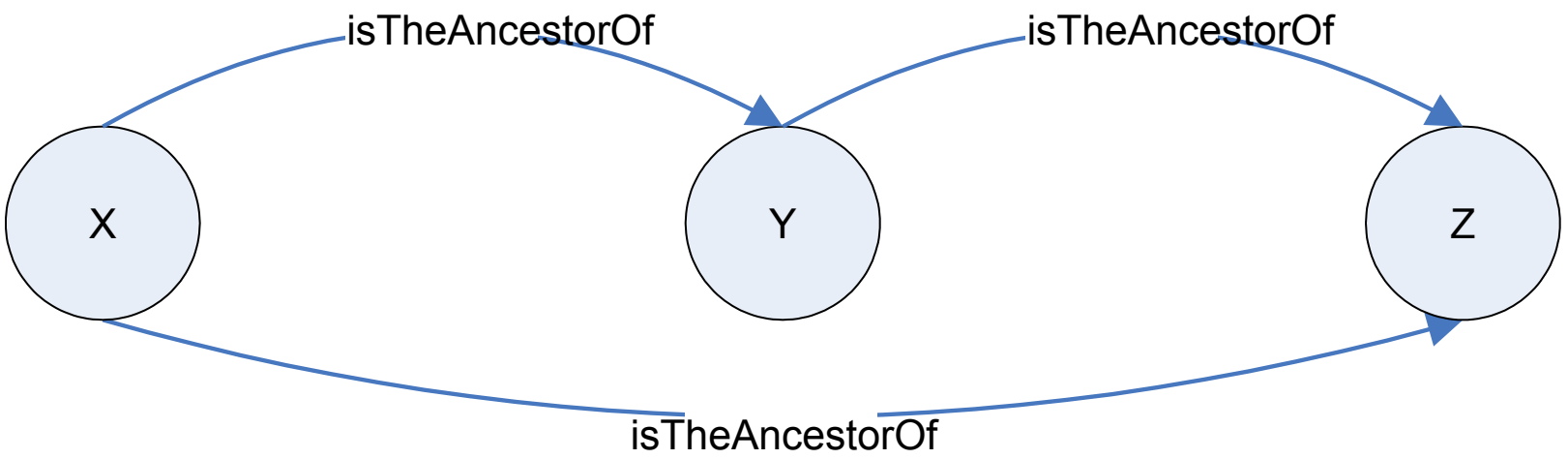
P1(X ,Y) iff P2(Y , X)



X isParentOf Y iff Y isSonOf X

Transitive Property

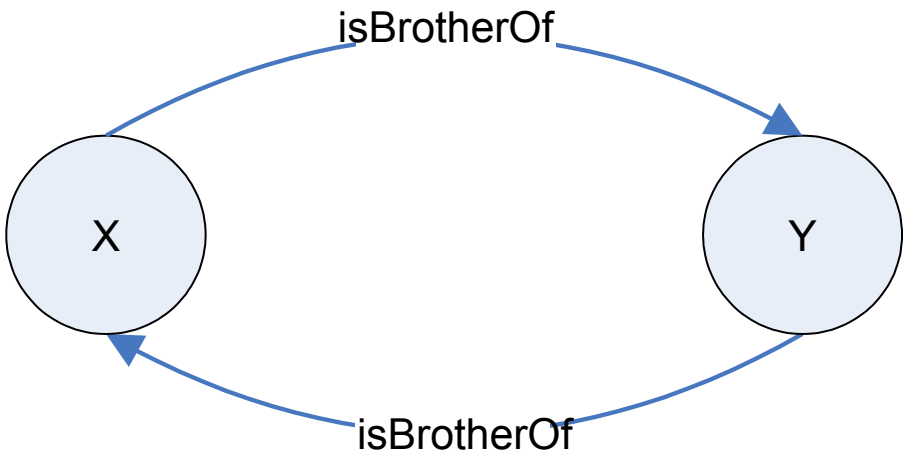
If P(X,Y) and P(Y,Z) then P(X,Z)



X isTheAncestorOf Y, Y isTheAncestorOf Z, then X isTheAncestorOf Z

Symmetric Property

P(X,Y) iff P(Y,X)



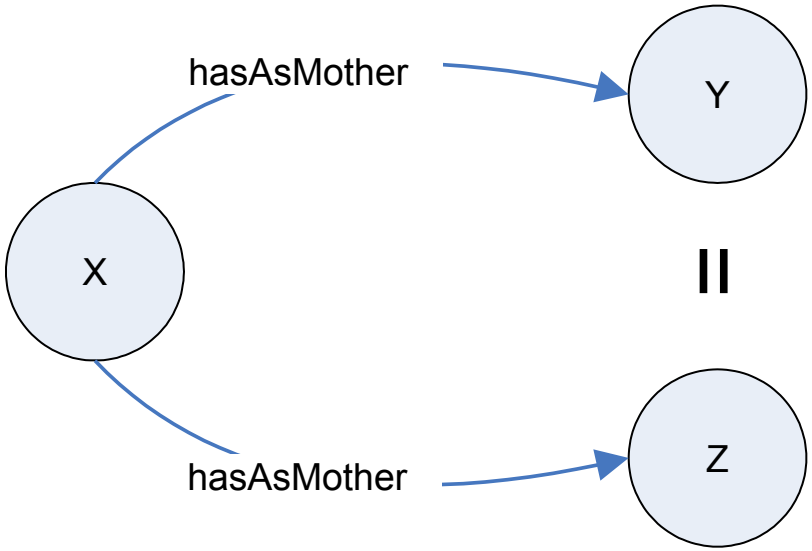
X isBrotherOf Y iff Y isBrotherOf X

Functional Property

Unicity

Only one instance can be linked

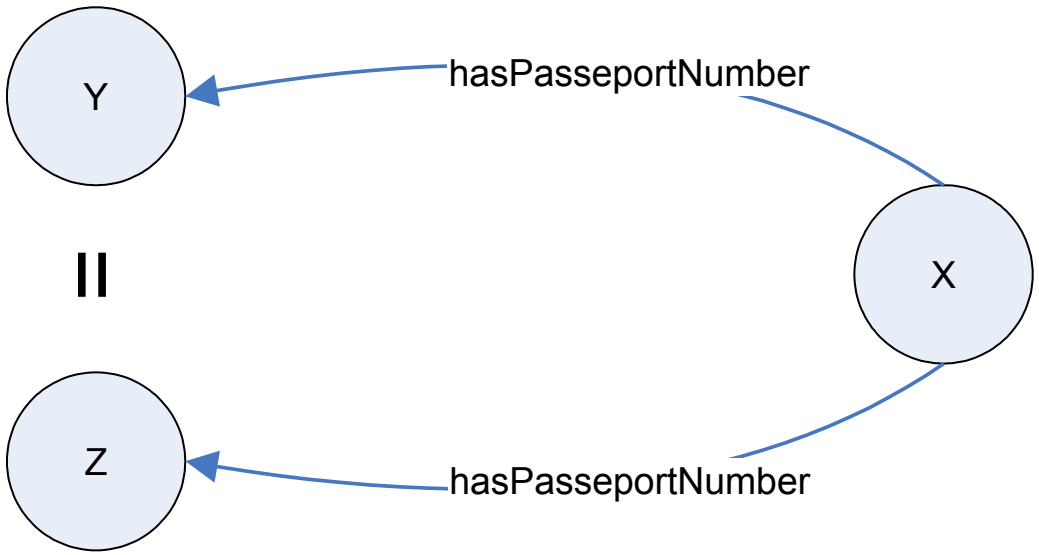
If P(X,Y) and P(X,Z) then Y=Z



- If X hasAsMother Y and X hasAsMother Z then Y=Z

Inverse Functional Property

P(Y,X) and P(Z,X) then Y=Z



Y hasPasseportNumber X and Z hasPasseportNumber X then Y=Z

OWL Class Constructors

Constructor	DL Syntax	Example	FOL Syntax
intersectionOf	$C_1 \sqcap \ldots \sqcap C_n$	Human $\sqcap$ Male	$C_1(x) \wedge \ldots \wedge C_n(x)$
unionOf	$C_1 \sqcup \ldots \sqcup C_n$	Doctor $\sqcup$ Lawyer	$C_1(x) \vee \ldots \vee C_n(x)$
complementOf	$\neg C$	$\neg$Male	$\neg C(x)$
oneOf	$\{x_1\} \sqcup \ldots \sqcup \{x_n\}$	{john} $\sqcup$ {mary}	$x = x_1 \vee \ldots \vee x = x_n$
allValuesFrom	$\forall P.C$	$\forall$hasChild.Doctor	$\forall y.P(x,y) \rightarrow C(y)$
someValuesFrom	$\exists P.C$	$\exists$hasChild.Lawyer	$\exists y.P(x,y) \wedge C(y)$
maxCardinality	$\leqslant nP$	$\leqslant$1hasChild	$\exists^{\leqslant n} y.P(x,y)$
minCardinality	$\geqslant nP$	$\geqslant$2hasChild	$\exists^{\geqslant n} y.P(x,y)$

TELECOM SudParis

What's Next ?

1. Hands-on Modeling in Protégé
    - Build class hierarchies
    - Declare roles (object properties & data properties)
    - Create individuals
    - Understand asserted vs inferred knowledge
2. Reasoning with OWL / Description Logics
    - Use class expressions and restrictions
    - Define necessary & sufficient conditions (EquivalentTo)
    - Run a reasoner (HermiT / Pellet) to:
        - Infer new facts
        - Detect inconsistencies
        - Validate modeling choices
3. Extending Expressiveness with SWRL when OWL alone cannot infer relationship patterns
4. Querying the Knowledge with SPARQL
    - Compare asserted vs inferred query results

Let's go to the lab!

TELECOM SudParis

Let’s go to the lab!